

บทที่ 3

ASP.NETเว็บแอปพลิเคชัน

3.1 ที่มาของ ASP

แอคทีฟเซิร์ฟเวอร์เพจ(ASP) ของบริษัทไมโครซอฟท์ คือเทคโนโลยีใหม่ในการสร้างเว็บเพจแบบไดนามิก หน้าของ ASP นั้น ประกอบด้วย HTML markup และสคริปต์ทางฝั่งเซิร์ฟเวอร์ ซึ่งจะประมวลผลตามอินพุตที่ได้รับเข้ามา ณ ขณะนั้น และสร้างโค้ด HTML ขึ้นมาจากการประมวลผล สคริปต์ทางฝั่งเซิร์ฟเวอร์จะทำงานเมื่อมีการร้องขอ (request) หน้า ASP ส่งมาถึงฝั่งเซิร์ฟเวอร์ อินพุตของหน้า ASP จะมาจากเบราว์เซอร์ทางฝั่งไคลเอนต์ผ่านทางเมธอด HTTP POST และ GET ASP มี อ็อบเจกต์โมเดลเพื่อความสะดวกขึ้นในการทำงานของผู้พัฒนา นอกจากการใช้อ็อบเจกต์จาก ASP อ็อบเจกต์โมเดล เช่น Application , Server , Request , Respond และ Session แล้ว ผู้พัฒนายังสามารถใช้ COM คอมโพเนนต์ อื่นๆที่มีอยู่บนเซิร์ฟเวอร์ ได้

ในการพัฒนาแอปพลิเคชันโดยใช้ ASP มักจะมีความยุ่งยากระหว่าง HTML markup และสคริปต์ทางฝั่งเซิร์ฟเวอร์ รูปแบบของหน้า ASP ทำให้เกิดความยุ่งยากในการจัดการและการนำมาใช้ใหม่ จึงมีการพัฒนาโดยเพิ่มไฟล์และฟังก์ชันในสคริปต์ทางฝั่งเซิร์ฟเวอร์ อย่างไรก็ตาม ความพยายามนี้ทำให้เกิดข้อเสียในด้านเวลา การจัดการเครือข่ายของไฟล์ที่เกี่ยวข้อง และยังมีผลกระทบทางด้านประสิทธิภาพจากการมี include ซ้อนกัน และการจัดการออบเจกต์ ไลต์ และ ขอบเขตของตัวแปร

ผู้พัฒนาที่ต้องทำงานที่เกี่ยวข้องกับการทำงานกับเบราว์เซอร์หลายตัวมักพบปัญหาในการทำเว็บแอปพลิเคชันให้สามารถทำงานได้กับหลายเบราว์เซอร์ซึ่งจะต้องคำนึงถึงความสามารถของเบราว์เซอร์ที่ไคลเอนต์ใช้ ซึ่งวิธีการแก้ไขปัญหามักจบลงด้วยการใช้แท็กHTMLที่เป็นแท็กพื้นฐานที่สุด และใช้สคริปต์ทางฝั่งไคลเอนต์เพื่อให้เป็นที่เข้าใจได้โดยทุกระบบเบราว์เซอร์บางครั้งมีความพยายามสร้างHTML marker สำหรับแต่ละเบราว์เซอร์โดยเฉพาะ ซึ่งได้ผลดีแต่เสียเวลาและต้องใช้ความพยายามอย่างมาก

การเขียนสคริปต์ใน ASP ใช้ภาษาได้จำกัดเช่น VB สคริปต์ และ Java สคริปต์เป็นต้น และสคริปต์ทางฝั่งเซิร์ฟเวอร์ จะถูกแปลทุกครั้งที่หน้าASPนั้นถูกเรียก ซึ่งไม่เป็นผลดีต่อประสิทธิภาพในการทำงาน นอกจากนั้นในเรื่องการจัดการกับสถานะ ใน ASP นักพัฒนาจะต้องทำทุกอย่างด้วยตนเอง โดยจะต้องเก็บ สถานะไว้ใน SQL เซิร์ฟเวอร์ หรือเนื้อที่เก็บข้อมูลภายนอกอื่นๆ จะเห็นได้ว่า

จะเห็นว่าแม้ASP จะเป็นเทคโนโลยีที่ดีในการสร้างเว็บเพจแบบไดนามิก แต่ก็ยังมีปัญหาอยู่อีกหลายด้าน จึงมีการแก้ไขและพัฒนามาเป็น ASP.NET

3.2 ที่มาของ ASP.NET

นักพัฒนาที่ใช้ Visual Basic มักจะคุ้นเคยกับความสะดวกสบายในการเขียนโปรแกรมโดยใช้ฟอร์มและคอนโทรลซึ่งมีรูปแบบที่สะดวก โดยลากสิ่งที่ต้องการลงมาไว้บนฟอร์มและเขียนโค้ดโดยใช้การเขียนโปรแกรมแบบ event-handling เนื่องจากวิธีนี้เป็นวิธีที่ไม่ยุ่งยาก จึงได้ถูกนำมาใช้ใน ASP.NET

ASP.NET ทำให้การพัฒนาเว็บไซต์สะดวกขึ้นด้วยการเขียนโปรแกรมโดยใช้ฟอร์ม ใน ASP.NET เราเรียกฟอร์มเหล่านี้ว่า เว็บฟอร์ม (Web Forms) ซึ่งนำมาแทนที่หน้า ASP การเขียนโปรแกรมแบบเว็บฟอร์มนี้เป็นแบบ event base เช่นเดียวกับใน visual basic นอกจากนี้ ASP.NET ยังแยกส่วนระหว่างส่วนแอปพลิเคชันและส่วนในการแสดงผลอีกด้วย

ASP.NET เกี่ยวข้องกับรูปแบบการเขียนโปรแกรม ASP โดยมีคุณสมบัติพิเศษเพิ่มเติมขึ้นมา ดังนี้

- ☐ มีการแยกส่วนเค็ดขาดระหว่างส่วนโค้ดการประมวลผลของแอปพลิเคชันและส่วนแสดงผล (HTML markup) จึงไม่มีความสับสนในโค้ดทั้งสองส่วนอีก
- ☐ มีกลุ่มของเซิร์ฟเวอร์คอนโทรลจำนวนมากให้เลือกใช้ ซึ่งจะถูกละเปลี่ยนเป็น HTML โดยอัตโนมัติ
- ☐ มีการจัดการสถานะโดยใช้ฮ็อบเจกต์เซสชัน
- ☐ เป็นรูปแบบการเขียนโปรแกรมแบบ event-base ทางฝั่งเซิร์ฟเวอร์ ทำให้ง่ายขึ้น
- ☐ สามารถเขียนโค้ดส่วนแอปพลิเคชันโดยใช้ภาษาใดใน Microsoft .NET ก็ได้ เช่น ภาษา VB, C# เป็นต้น โค้ดของแอปพลิเคชันบนฝั่งเซิร์ฟเวอร์จะถูกคอมไพล์เพื่อประสิทธิภาพการทำงานที่ดีกว่า
- ☐ มี Visual Studio.NET เป็นเครื่องมืออำนวยความสะดวกในการพัฒนาเว็บฟอร์ม

3.3 รู้จักกับ System.Web.UI Namespace

ในส่วนนี้จะอธิบายถึงคลาส ที่สำคัญที่อยู่ในเนมสเปซชื่อ System.Web.UI ในเฟรมเวิร์คของ ASP.NET เนมสเปซ System.Web.UI นั้นระบุคลาสและอินเทอร์เฟซที่ใช้ในการสร้างสิ่งต่างๆ ในคลาสเว็บฟอร์ม ที่สำคัญที่สุดใน System.Web.UI คือคลาส Control ซึ่งบอกคุณสมบัติ เมธอด และ event ของแต่ละเซิร์ฟเวอร์คอนโทรลในเฟรมเวิร์คของเว็บฟอร์ม อีกคลาสที่สำคัญคือเพจ ซึ่งจะกล่าวถึงในลำดับต่อไป

3.3.1 คลาส Control

คลาส Control เป็นรากฐานของคอนโทรลทั้งหมด ตัวอย่างเช่น textbox หรือ button ก็คือคอนโทรล โดยทั่วไปคลาส Control จะมีฟังก์ชันและคุณสมบัติของส่วนติดต่อกับผู้ใช้ทุกตัว ทุกอย่างที่เราเห็นใน ASP.NET ก็เป็นคลาสที่สืบทอดมาจากแหล่งใดแหล่งหนึ่งทั้งสิ้น

3.3.1.1 คุณสมบัติของ Control

คลาส Control มีคุณสมบัติที่สำคัญดังต่อไปนี้ Control, ID, Parent, EnableViewState, Visible, Context และ ViewState คุณสมบัติของคอนโทรลเหล่านี้แทนลูกของ instance ของคอนโทรลคุณสมบัติของคลาสแม่จะบอกคลาส แม่ของคอนโทรล คุณสมบัติทั้งสองนี้ ทำให้มีลำดับชั้น(hierarchy)ของคอนโทรลในหน้าเว็บคุณสมบัติ ID ทำให้คอนโทรลถูกเข้าถึงจากการเขียนโปรแกรมได้โดยใช้ IDตามด้วยจุดและชื่อคุณสมบัติหรือเมธอดอื่นๆที่เราต้องการจะเข้าถึงของคอนโทรลนั้น เช่น MyObjectId.propertyname และยังทำให้เราสามารถเขียนตัวจัดการกับเหตุการณ์ที่เกิดขึ้นกับคอนโทรลได้ด้วย

EnableViewState เป็น flag ที่บอกว่าคอนโทรลจะยังคงเก็บview ของสถานะมันไว้หรือไม่ เช่นเดียวกับ view สถานะทั้งหมดของคอนโทรลลูกของมัน คุณสมบัติของ Context ทำให้เราสามารถดึงข้อมูลของ HTTP request ได้

ViewState เป็นอินสแตนซ์ของคลาส StateBag ซึ่งใช้ในการเก็บชื่อและค่าของข้อมูลที่สามารถสร้างให้เข้าถึงได้โดยหลาย request สำหรับหน้าเพจเดียว ชื่อและค่าเหล่านี้คืออินสแตนซ์ของคลาส StateItem

3.3.1.2 เมธอดในคลาส Control

เมธอดของคลาส Control นั้นมีเป็นจำนวนมาก อย่างไรก็ตามเราจะกล่าวถึงเพียงเมธอดที่มีความสำคัญ

เมธอด DataBind

เป็นการโยงคอนโทรลเข้ากับแหล่งข้อมูล เมื่อเมธอด นี้ถูกเรียก แท็กของการผูกข้อมูล <%#%> จะถูกนำมาพิจารณาใหม่เพื่อให้ข้อมูลใหม่ถูกโยงเข้ากับตำแหน่งtagที่เหมาะสม

เมธอด CreateChildControls

จะถูกเรียกก่อนที่คอนโทรลcompositional custom จะถูกจัดแสดงคอนโทรล compositional custom นั้นคล้ายกับคอนโทรล ActiveX คือมันจะจัดการสร้างคอนโทรลอื่นๆ ในการพัฒนาคอนโทรล custom เมธอดนี้สามารถถูกoverride เพื่อให้ผู้พัฒนาคอนโทรล custom สามารถสร้าง คอนโทรลลูก ก่อนทำการแสดงคอนโทรลไม่ว่าในการทำครั้งแรกหรือครั้งต่อไปก็ตาม

เมธอด Render

คล้ายกับ CreateChildControls เริ่มต้นใช้สำหรับพัฒนาคอนโทรล custom ผู้พัฒนาคอนโทรลจะ overrideเมธอดนี้ เพื่อแสดงตัวเนื้อหาในคอนโทรลผ่านทางพารามิเตอร์ HtmlTextWriter ที่เตรียมไว้

เมธอด SaveViewState และ LoadViewState

เก็บบันทึกและเรียกใช้งานของคอนโทรล คอนโทรลของเซิร์ฟเวอร์ คงสถานะของมันระหว่างการrequest ผ่านทางเมธอด นี้

3.3.2 คลาสเพจ

คลาสเพจนี้สืบทอดมาจากคลาส Control ซึ่งหมายความว่ามันสืบทอดมาทั้งคุณสมบัติเมธอด และ event ที่แสดงโดยคลาส Control นอกจากการสืบทอดแล้วคลาส เพจ ระบุคุณสมบัติเมธอด และ event ที่เจาะจงมากขึ้น สำหรับหน้าเว็บในเฟรมเวิร์คของ ASP.NET

ใน ASP.NET ทั้ง Application , Request , Respond , Server และ Session เป็นคุณสมบัติของคลาสเพจ นอกเหนือจากอ็อบเจกต์เหล่านี้คลาส เพจ ยังมีคุณสมบัติอื่นๆอีก เช่น Cache, ErrorPage, Postback , IsValid, Trace และ Validators

3.3.2.1 ตัวอย่างคุณสมบัติและเมธอดของคลาสเพจได้แก่

คุณสมบัติ Cache

ซึ่งไปยังอ็อบเจกต์ cache ของขอบเขตของหน้าปัจจุบัน ในที่นี้ ทรัพยากรเช่นการติดต่อฐานข้อมูล(database connection)จะถูกเก็บเพื่อนำมาใช้ใหม่โดยไม่ต้องสร้างการติดต่อใหม่ หากว่า cache item นั้นยังไม่หมดอายุ

คุณสมบัติ ErrorPage

บอกหน้าที่จะถูกเรียกให้แสดงเมื่อมีerror เกิดขึ้น อาจใช้ @Pageแทนได้

คุณสมบัติ IsPostBack

บอกว่าการเรียกหน้าเว็บเป็นการโหลดครั้งแรกหรือไม่ หาก IsPostBack มีค่าเป็น true แสดงว่าไม่ใช่การโหลดครั้งแรก จึงไม่ควรทำการ initialize หน้าเพจซ้ำ เพื่อให้ประสิทธิภาพดีขึ้น

คุณสมบัติ Validator

รวมกลุ่มของเซิร์ฟเวอร์คอนโทรลซึ่งสามารถกำหนดระยะเวลาการทำงานของตัวเองลงในคุณสมบัติ Validatorของหน้าเว็บ เมื่อหน้าเว็บใดต้องมีการกำหนดอายุของตัวเอง ก็จะกำหนดอายุของคอนโทรลทั้งหมดในหน้านั้นแล้วกำหนดค่าที่เหมาะสมให้แก่คุณสมบัติ IsValid

คุณสมบัติ Trace

อ้างอิงจากอ็อบเจกต์ `TraceContext` ซึ่งใช้บอกค่าเตือนหรือข้อความบอกความผิดพลาดที่เกิดขึ้น สามารถกำหนดให้ทำการตรวจสอบหรือไม่ทำก็ได้โดยกำหนดใน `web.config` ซึ่งเป็นเท็กซ์ไฟล์ที่เก็บค่าขณะรันไทม์ของแอปพลิเคชันของ ASP.NET

3.4 ไวยากรณ์สำหรับ Web Form

3.4.1 ไคเร็กทีฟ

สำหรับ ASP แบบเก่า ไคเร็กทีฟจะมีรูปแบบเป็น `<%@attribute= value%>` เพราะว่าไคเร็กทีฟใน ASP นั้นมีเพียงแบบเดียวเท่านั้น

ใน ASP.NET ได้มีการเพิ่มไคเร็กทีฟใหม่ซึ่งได้แก่ `Page`, `Control`, `Import`, `Register`, `Assemble` และ `OutputCache` และไวยากรณ์ใหม่ที่น่าสนใจคือ `<%@ directive [attribute=value] %>` โดยถ้าการประกาศกระทำโดยใช้ไวยากรณ์แบบเก่าคือไม่มีการใส่ไคเร็กทีฟ ก็จะหมายถึง ประกาศให้เป็นไคเร็กทีฟแบบปกติซึ่งก็คือ `Page`

3.4.1.1 @Page

ใน ASP.NET นอกจากจะประกอบด้วยไคเร็กทีฟแบบ ASP เก่าแล้ว ยังสนับสนุน attribute ที่สำคัญ เช่น `ErrorPage`, `Inherits`, `Src`, และ `EnableViewState` อีกด้วย ตัวอย่างการประกาศใช้งาน

```
<%@ Page Language="VB" ErrorPage = "URL" EnableViewState = "true"%>
```

3.4.1.2 @Control

เช่นเดียวกันกับ `Page` ไคเร็กทีฟ จะมีการประกาศใช้ในไฟล์ (.aspx) สำหรับ `Control` ไคเร็กทีฟนั้นจะใช้ใน ASP.NET `Control` คือประกาศในไฟล์ (.ascx)

3.4.1.3 @Import

เราสามารถใช้ `Import` ไคเร็กทีฟ ในการอ้างถึง เนมสเปซที่จะนำมาใช้ในหน้านั้นๆ ซึ่งโปรแกรมที่เราสร้างขึ้นจะสามารถเข้าถึงทุกๆ คลาสที่มีอยู่ในเนมสเปซนั้น ตัวอย่างเช่นถ้าเราต้องการใช้งาน ADO.NET เราต้องใส่ไคเร็กทีฟดังนี้คือ

```
<%@ Import Namespace="System.Data" %>
```

```
<%@ Import Namespace="System.Data.OleDb" %>
```

3.4.1.4 @Register

เป็นไคเร็กทีฟที่ใช้ ลงทะเบียนการใช้งานเซิร์ฟเวอร์คอนโทรลในหน้า `Page` นั้นๆ ตัวอย่างเช่น

```
<%@ Register Tagprefix = "Ch07"
```

TagName = "MyCustomControl"

Src = "MyCustomControl.ascx" %>

โดยชื่อของคอนโทรลคือ MyCustomControl และ คำนำหน้าเมื่อการประกาศใช้คอนโทรลคือ Ch07
ที่อยู่ของคอนโทรลคือ MyCustomControl.ascx

3.4.1.5 @Assembly

เป็นไคเร็กทีฟที่ใช้บ่งบอกว่า Page นั้นๆ เป็นของ assemble ไคตัวอย่างเช่น

<%@ Assembly Name = "Ch07" %>

หมายความว่า โปรแกรมในหน้า Page นี้สามารถเข้าถึงอะไรก็ได้ใน assembly Ch07

3.4.1.6 @OutputCache

เราสามารถใช้ Output Cache ไคเร็กทีฟในการควบคุม ระยะเวลาของ output caching ระหว่างที่หน้า Page นั้นเปิดขึ้น

3.4.2 ขอบเขตการประกาศคำสั่ง

เช่นเดียวกับ ASP ในการประกาศขอบเขตคำสั่งทำงาน จะหมายถึงการกำหนด ขอบเขตคำสั่งที่จะถูกประมวลผลและสั่งให้ทำงานในหน้า Page นั้นๆ ซึ่งในขอบเขตนั้นจะมี คุณลักษณะที่เรียกว่า "runat" ซึ่งเป็นการใช้กำหนดว่า ขอบเขตของคำสั่งเหล่านี้ จะประมวลผลที่ฝั่งไคลเอนต์ หรือ ฝั่งเซิร์ฟเวอร์ ซึ่งถ้าต้องการให้มีการทำงานที่ฝั่งเซิร์ฟเวอร์ จะต้องมีการกำหนดให้มี คุณลักษณะเป็น "server" และถ้าไม่มีการกำหนดจะหมายความว่าให้ทำงานในฝั่งไคลเอนต์ ตัวอย่างเช่น

```
<script runat="server" [language="codelanguage"]>
```

```
    'code
```

```
</script>
```

สำหรับการทำงานทั้งสองแบบเราสามารถกำหนดที่อยู่ของคำสั่งได้ โดยการทำเช่นนี้เป็นการแยกส่วนของคำสั่งออกจากส่วนของเอกสาร HTML การประกาศกระทำโดย

```
<script runat="server"
```

```
    [language="codelanguage"]
```

```
    [src="externalfilename"]/>
```

3.4.3 HTML-Control Syntax

HTML คอนโทรลมีความคล้ายกับมาตรฐาน HTML ปกติอยู่มาก จะแตกต่างกันที่มีการเพิ่มการประกาศ "id" และ "runat" แอททริบิวต์ ซึ่งโดยปกติการใช้ id กับการเขียนโปรแกรมนั้นจะเกี่ยวกับการเขียนใน

ลักษณะ Dynamic HTML จะแตกต่างกันตรงที่ HTML คอนโทรล จะเป็นลักษณะของ คอนโทรลที่มีทำงานในฝั่งเซิร์ฟเวอร์ ตัวอย่างเช่น

```
<input id="cmd1" runat="server"
type="button" value="Click Me" />
```

โดยสังเกตว่าการประกาศต้องเป็นแบบ server เพราะว่าเว็บฟอร์มจะมีการใช้ POST เมธอดในการควบคุมเรื่องสถานะการทำงาน

เมื่อทำการประมวลผลคำสั่ง ASP.NET จะทำการสร้างสคริปต์ขึ้นจากออปเจ็ค HtmlControl จากตัวอย่างข้างต้น ASP.NET จะทำการสร้างคอนโทรลขึ้นจาก HtmlInputButton โดยมี id คือ cmd1

นอกจากนี้ยังสามารถควบคุม อีเวนต์ของคอนโทรลได้อีกด้วย โดยการประกาศจะกระทำผ่าน HTML Tag ตัวอย่างเช่น ถ้าต้องการควบคุมอีเวนต์ onclick สามารถเพิ่มในส่วนของ HTML Tag ดังนี้

```
Onserverclick = "handleServerClick"
```

3.4.4 Custom-Control Syntax

ในส่วนของ คอนโทรลที่กำหนดเอง ก็จะคล้ายกับ HTML เซิร์ฟเวอร์คอนโทรล โดย Custom คอนโทรลจะมีส่วนของ id และ runat แอททริบิวต์ แต่อย่างไรก็ตาม Custom คอนโทรลก็ไม่เหมือนกับ HTML มาตรฐาน ในการที่จะใช้งานเราสามารถประกาศตามรูปแบบดังนี้คือ

```
< tagprefix:tagname id="controlID" runat="server" eventname="eventHandler" />
```

ในการเรียกใช้งาน จะต้องใส่ prefix เป็น asp ดังตัวอย่างต่อไปนี้

```
<asp:TextBox id=txt1 runat=server></asp:TextBox>
```

3.4.5 Data-Binding Expressions

การผูกข้อมูลหรือ Data Source เข้ากับคอนโทรลใดๆ เราสามารถทำได้ตามไวยากรณ์ดังนี้คือ

```
<%# data-binding-expression %>
```

โดยการใช้ Data Bind นั้น Web Form จะไม่ทำการ Bind ให้อัตโนมัติ เราจะต้องทำการใส่เมธอด

DataBind() เองภายในโค้ดโปรแกรม

ตัวอย่างการใช้ Data Binding ดังนี้คือ

```
<asp:Label text='<%# TestData %>' runat=server/>

<html>

    <head><title>Data Binding</title></head>

    <body>

        <script language="C#" runat= server>

            /*declare the variable we want to bind to.

            Public string TestData;

            Void Page_Load(Object oSender, EventArgs oEvent) {

                TestData = "HelloWorld";

                Page.DataBind();

            }

        </script>

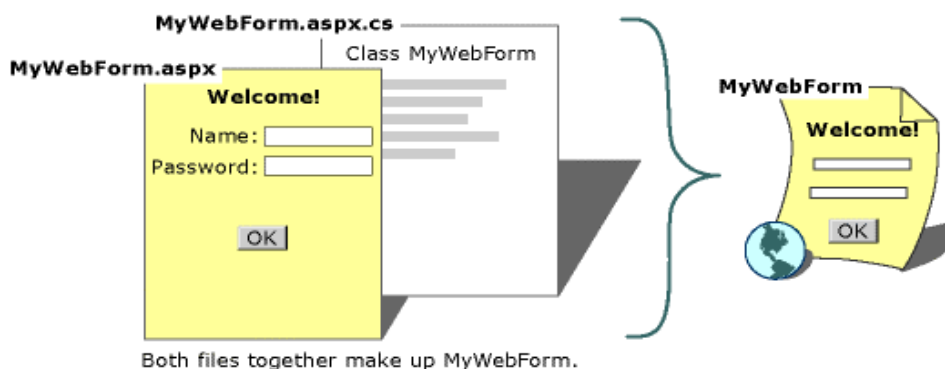
    </body>

</html>
```

3.5 การพัฒนาเว็บแอปพลิเคชันด้วย ASP.NET

3.5.1 ส่วนประกอบคอมโพเนนต์ของ Web Form

ใน Web Forms pages นั้น การเขียน program ของ user interface แบ่งเป็นสองส่วนคือ visual Component และ logic ซึ่งก็คล้ายกับการที่ Visual Basic มีการแบ่งระหว่างส่วนของ form และส่วน code เบื้องหลังที่ form นั้นทำงานด้วยนั่นเอง ส่วนแสดงผลที่มองเห็นได้นั้นก็คือ Web Form page ซึ่งประกอบด้วยไฟล์ ที่มี HTML หรือ ASP.NET server control. Web Form page ทำงานเหมือนเป็น container สำหรับ static text และส่วนควบคุมที่เราต้องการแสดงผล ในส่วน logic ของ Web Form page ประกอบด้วย โค้ด ที่เรา สร้างขึ้นเพื่อสื่อสารกับ form logic การเขียนโปรแกรม จะอยู่แยกกับส่วน file ของ user interface ซึ่งไฟล์นี้จะเปรียบเสมือน โค้ดที่อยู่เบื้องหลังการทำงาน สามารถเขียนด้วยภาษา Visual Basic หรือ Visual C#



รูปที่ 3-1 ส่วนประกอบของเว็บฟอร์ม

ไฟล์ที่ประกอบด้วยคลาสต่างๆและ เบื้องหลังการทำงาน สำหรับ Web Form ทั้งหมดจะถูก คอมไพล์ เป็น ไฟล์ dynamic-link library(.dll) page ที่ เป็น .aspx ก็จะถูก compile เช่นกัน แต่มีกระบวนการที่ต่างกัน ในครั้งแรกที่ ผู้ใช้ เรียก file หน้า .aspx ASP.NET จะสร้างไฟล์ .NET class มาเป็นตัวแทนของ page โดยอัตโนมัติ แล้ว คอมไพล์ เป็น ไฟล์ .dll ตัวที่สอง class ที่ถูกสร้างสำหรับ .aspx page สืบทอดคุณสมบัติมาจาก class ที่เป็น โค้ดการทำงานเบื้องหลังที่ถูก compile ไปแล้วก่อนหน้านี้ เมื่อผู้ใช้ร้องขอ URL ของ web page ไฟล์ .dll จะ run บน server และสร้างผลเป็น HTMLแบบไดนามิกสำหรับ page ที่เรียก

3.5.2 วงจรชีวิตของ Web Form

ใน ASP เว็บเพจจะเริ่มต้นวงจรชีวิตเมื่อไคลเอนต์เริ่มทำการร้องขอเพจที่เจาะจงมา IIS ทำการกระจายคำและทำการใช้งานสคริปต์บน ASP เพจ เพื่อทำให้เป็นเนื้อความ HTML ในที่สุดทำการแปลงก็จะสิ้นสุดอายุของเพจนั่นก็จะสิ้นสุด ถ้าคุณมีฟอร์มที่ส่งข้อมูลกลับไป ASP เพจก็จะเริ่มกระบวนการใหม่ เหมือนกับ ASP กระทำคำร้องใหม่ โดยไม่ต้องรู้เกี่ยวกับขั้นตอนก่อนหน้านี้เลย การส่งผ่านข้อมูลกลับไปเพจเริ่มต้นสำหรับการประมวลผลก็เป็นการถูกอ้างแบบโพสต์แบ็ก(postback)ด้วย

ใน ASP.NET มีหลายสิ่งที่แตกต่างกันเล็กน้อย เพจยังคงเริ่มที่การร้องขอของไคลเอนต์อย่างไรก็ตาม มันจะยังคงอยู่เรื่อยๆเท่าที่ไคลเอนต์ยังคงติดต่อกับเพจออยู่ เพื่อทำให้ง่ายขึ้นอาจจะกล่าวได้ว่าเพจยังคงอยู่ไปเรื่อยแต่ในความเป็นจริงมีเพียงขั้นตอนการดู(view)ของเพจระหว่างการร้องขอเท่านั้น ขั้นตอนการดูเท่านั้นที่อนุญาตให้ความคุมบนเซิร์ฟเวอร์ที่จะปรากฏว่า ถ้ามันยังคงครองอี่เอนต์ของเซิร์ฟเวอร์ เราสามารถค้นหาขั้นตอนการโพสต์แบ็กของเพจผ่านคุณสมบัติ IsPostBack ของเพจอ็อบเจ็กต์และการกำหนดค่าเริ่มต้นใหม่ การครองอี่เอนต์ระหว่างการโพสต์แบ็กมันก็คือการทำ ASP.NET แตกต่างกว่าการพัฒนา ASP ตามแบบธรรมดาอย่างมาก

จากตัวอย่างเราทำการขยายส่วนต้น เช่นการครองโพสค์แบ็ค เมื่อทำการโหลดอีเวนต์ที่ถูกถือครองครั้งแรก เราจะทำการย้ายข้อมูลลงใน drop-down list box หลังจากนั้นเราจะระบุเวลาของอีเวนต์ที่เพิ่มขึ้นโดยไม่มีการโหลดข้อมูลเข้ามาใหม่เท่านั้น ตัวอย่างนี้แสดงให้เห็นว่าเซิร์ฟเวอร์อีเวนต์แฮนด์เลอร์(handler)ที่ชื่อ handlerButtonClick เป็นขอบเขตของอีเวนต์ที่ชื่อ ServerClick ของปุ่ม

```
<html>

<head><title>Testing Page Event</title></head>

<body>

    <script language="C#" runat=server>

        void Page_Init(Object oSender,EventArgs oEvent){

            labelInit.Text = DateTime.Now.ToString( );

        }

        void Page_Load (Object oSender, EventArgs oEvent) {

            labelInit.Text = DateTime.Now.ToString( );

            if (!IsPostBack){

                selectCtrl.Items.Add("Acrua");

                selectCtrl.Items.Add("BMW");

                selectCtrl.Items.Add("Cadillac");

                selectCtrl.Items.Add("Mercedes");

                selectCtrl.Items.Add("Porsche");

            }else{

                labelLoad.text += "(Postback)";

            }

        }

        void handleButtonClick(Object oSender, EventArgs oEvent){

            labelOutput.Text = "You've selected: "+ selectCtrl.Value;

            labelEvent.Text = DateTime.Now.ToString( );

        }

    </script>
```

```

<form runat = server>

    Init Time: <asp:Label id=labelInit runat=server/><br/>
    Load Time: <asp:Label id=labelLoad runat=server/><br/>
    Event Time: <asp:Label id=labelEvent runat=server/><br/>
    Choice: <select id=selectCtrl runat=server/><br/>
    <asp:Label id=labelOutput runat=server/></select><br/>
    <input type=button value=update
        onServerClick="handleButtonClick" runat=server />

</form>

</body>

</html>

```

วงจรชีวิตของเว็บฟอร์ม ประกอบด้วยสามขั้นตอนหลัก การกำหนดรูปแบบ การถือครองอีเวนต์ และการสิ้นสุด โดยขั้นตอนเหล่านี้จะข้ามผ่านคำร้องขอที่เหมือนกันซึ่งแข่งกับนโยบาย serving-one-page-at-a-time ที่พบใน ASP

3.5.2.1 การกำหนดรูปแบบ

ขั้นตอนการกำหนดรูปแบบนี้ โหลดอีเวนต์ของเพจจะถูกยกขึ้นมามีงานของคุณในการถือครองอีเวนต์นี้เพื่อติดตั้งเพจคุณ เพราะว่าโหลดอีเวนต์จะถูกยกขึ้นเมื่อการควบคุมพร้อมแล้วงานคุณตอนนี้คือการอ่านและทำการอัปเดตคุณสมบัติการควบคุมเหมือนเป็นส่วนของการติดตั้งเพจ ในตัวอย่างโค้ดข้างต้น เราจะถือครอง โหลดอีเวนต์เพื่อย้ายข้อมูลบางอย่างลง drop-down list เราอัปเดตข้อความของการควบคุม labelLoad เพื่อแสดง เวลาโหลดอีเวนต์ที่เกิดขึ้น ในแอปพลิเคชันของคุณคุณจะต้องข้อมูลที่เป็นไปได้จาก ฐานข้อมูลและทำการหนดค่าเริ่มต้นฟอร์มฟิลด์ด้วยค่าดีฟอลต์

คุณสมบัติ IsPostBack ของเพจแสดงว่าเพจแรกได้ถูกโหลดหรือมันเป็นการโพสต์แบคเช่นถ้าคุณควบคุมเนื้อหารายการของข้อมูลคุณจะต้องโหลดการควบคุมเพจแรกที่ถูกโหลดโดยการตรวจสอบคุณสมบัติ IsPostBackของเพจนี้นั้นเมื่อ IsPostBack มีค่าเป็นจริงคุณจะรู้ว่ารายการควบคุมอ็อบเจกต์ได้ถูกโหลดด้วยข้อมูลเรียบร้อยแล้ว ไม่จำเป็นที่ต้องย้ายข้อมูลในรายการใหม่ตัวอย่างโค้ดข้างต้นเราจะข้ามการย้ายข้อมูลของ drop-down และแสดงเพียง สตริงว่า “(postback)”

คุณอาจจะต้องกระทำการผูก(bind)ข้อมูลและกำหนดค่าใหม่ data-binding expressions ในตอนแรก และลำดับต่อมาของเพจนี้นี้

3.5.2.2 การถือครองอีเวนต์

ในขั้นตอนกลางนี้ ฟังก์ชัน server event-handlingของเพจจะถูกเรียกเป็นผลของบางอีเวนต์เป็นการถูกเรียกให้ทำงานจากฝั่งไคลเอนต์ อีเวนต์เหล่านี้มาจากการควบคุมที่คุนแทนที่บนเว็บฟอร์ม

3.5.2.3 การจบการทำงาน

ที่ขั้นตอนนี้เพจจะสิ้นสุดการแปลงและถูกกำจัดทิ้งแล้วสิ่งที่คุนต้องรับผิดชอบคือการทำความสะอาดไฟล์แฮนด์เลอร์ ยกเลิกการติดต่อกับฐานข้อมูลและปล่อยอ็อบเจกต์ถึงแม้คุนจะเชื่อมั่นการทำงานของตัวกำจัดขยะ(garbage collection)ของ CLR ก็ตามแต่เราขอยืนยันอย่างยิ่งยวดว่าควรจำทำความสะอาดด้วยตัวเองเพราะตัวกำจัดขยะทำงานเป็นระยะๆเท่านั้น บนการทำงานหนักๆถ้าตัวกำจัดขยะไม่เหมาะสมจะมีทรัพยากรที่ไม่ได้คืนให้ระบบทำให้หน่วยความจำหมดไปได้และทำให้ระบบคุนค้างไป

เราสามารถทำความสะอาดตัวอย่างข้างต้นได้ด้วยการอันโหลดอีเวนต์แฮนด์เลอร์ดังที่จะแสดงต่อไปนี้เพราะว่าไม่มีแสดงในตัวอย่างเราจะแสดงเฉพาะฟังก์ชันเป็น เทมเพลต

```
Void Page_Unload(Object oSender,EventArgs oEvent){
    //cleaning up code here
}
```

3.6 การจัดการ State และ Scalability

3.6.1 ASP.NET Session-State Management

ASP.NET ได้ปรับปรุงการจัดการเรื่องสเททใน ASP เดิม โดยปรับปรุงมาเป็นลักษณะโมเดลแบบ out-of-process โดยการจัดการสเททจะมองลักษณะของฟาร์มของ Web Server โดยการสั่งให้มีการสลับไปยังเพจใดๆ ในฟาร์มจะไม่เกินการหายของเซสชัน

ในการเปลี่ยนแปลงค่าเกี่ยวกับการจัดการการบริหาร session ใน ASP.NET เราสามารถกระทำผ่านไฟล์ที่ชื่อว่า web.config โดยความเป็นจริงแล้วเราสามารถทำการกำหนดค่าได้สองทางได้แก่ทางระดับ แมทชีน และระดับ แอปพลิเคชัน โดยในระดับแมทชีนสามารถกำหนดผ่านไฟล์ที่อยู่ในที่

WinNT\Microsoft. NET\Framework\<version>\CONFIG\machine.config ในขณะที่ในระดับแอปพลิเคชันสามารถกำหนดผ่าน web.config ที่อยู่ในไดเรกทอรีเดียวกับแอปพลิเคชัน

โดยโค้ดที่อยู่ในไฟล์ web.config มีดังนี้คือ

```
<configuration>
  <system.web>
    <sessionstate
      mode="Inproc"
      cookieless="false"
      timeout="20" />
  </system.web>
</configuration>
```